

Contents

1	Symbols	3
2	Deep Learning(COMP838)	5
2.1	Terms Explain	5
2.1.1	LSTM(Long Short-Term Memory)	5
2.1.2	GRU(Gated Recurrent Unit)	6
2.1.3	LSTM vs GRU Comparison	6
2.1.4	AlexNet	6
2.1.5	R-CNN (Region-based CNN)	7
2.1.6	YOLO (You Only Look Once)	7
2.1.7	SSD(Single Shot MultiBox Detector)	8
2.1.8	GPT (Generative Pre-trained Transformer)	8
2.1.9	Reinforcement Learning	8
2.1.10	Transfer Learning	9
2.1.11	SVM(support Vector Machine)	9
2.1.12	Ensemble Learning	9
2.2	What is deep learning	10
2.2.1	Neural Network	10
2.3	Matlab	12
2.3.1	Data Labelling and Augmentation	12
2.3.2	Implementation of Machine Learning	12
2.4	Confusion Matrix	12
2.5	Artificial Neural Networks	13
2.5.1	Three Functional Operations	13
2.5.2	Feedforward Neural Networks	13
2.5.3	Multilayer Perceptron	13
2.5.4	Loss Functions	13
2.5.5	Three Datsets	14
3	Data Mining and Machine Learning(COMP809)	15
3.1	How to use Jupyter Notebook in VSC	15
3.2	How to install Jupiter Notebook without VSC	15
3.3	Regression Models	15
3.3.1	Simple Linear Regression	15
3.3.2	Parameter estimation – least squares	16
3.3.3	Model assumptions	16
3.3.4	Multiple Linear Regression	17
3.3.5	others	18
3.4	Non-numeric input coding	18
3.5	Curse of Dimensionality	18
3.5.1	Prerequisite	18
3.5.2	Feature Selection	19
3.5.3	Feature Extraction	19
4	Statistical Programming for Data Sciences(COMP828)	25
4.1	Descriptive Statistics	25
4.2	Plots	25
4.3	mode function in R	26

5	Video and Image Processing(COMP820)	27
5.1	Self-directed Learning: Matlab Resources for Image Enhancements	27
5.2	Homework and self-paced learning courseware	27
5.2.1	MatLab image processing workflow	27
5.2.2	Conclusion	31
5.2.3	demo	31
5.3	Lecture 3	32
5.4	Other Learning Resources	32
6	Generative AI(COMP7??)	33
6.1	Transformer-Based Models (LLMs)	33
6.2	GANs (Generative Adversarial Networks)	33
6.3	Diffusion Models	33
6.4	VAEs (Variational Autoencoders)	34
6.4.1	Important Correction First	34
6.4.2	Step-by-Step: What Happens After Compression	34
7	Mathamatics	35
7.1	norm	35
7.2	Harmonic Mean	35
7.3	Marginal Probability	35
7.4	Conditional Probability	36

Chapter 1

Symbols

Term	Description
XOR	Exclusive OR logical operation
Confidence level	Probability that a confidence interval contains the true parameter
$\tanh$	Hyperbolic tangent function: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
OLS	Ordinary Least Squares regression method
Variance	Measure of how data points spread around the mean: $\sigma^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$
Covariance	variance between two different data
Standard deviation	Square root of variance: $\sigma = \sqrt{\sigma^2}$
Outlier	Observation that deviates significantly from other data points
Correlation	Measure of linear relationship between two variables: $r = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$
Residuals	Difference between observed and predicted values in a model: $e_i = y_i - \hat{y}_i$
NLP	Natural Language Processing
CV(Coefficient of Variation)	$CV = \frac{\sigma}{\mu}$
RSS	Residual Sum of Squares OR sum of squared residuals.
Regularization	This technique is used to prevent overfitting by adding a penalty term to the loss function (e.g., L1 or L2 regularization).
Normalization	ypically refers to scaling data to a fixed range, most commonly between 0 and 1.
Standardization	Refers to rescaling data so that it has a mean of 0 and a standard deviation of 1 (often called Z-score normalization).

Chapter 2

Deep Learning(COMP838)

2.1 Terms Explain

Table 2.1: Relations between models

CNN	AlexNet(Deep Learning Revolution)	2012
	R-CNN	2013
	ResNet	2015
	GoogLeNet	2015
	SSD	2015
	YOLO	2016
RNN	LSTM	1997
	GRU	2014
GAN(2014)		
Transformer(2017)	GPT	2018

In deep learning, **LSTM** and **GRU** are special types of Recurrent Neural Networks (RNNs) designed to handle long-term dependencies in sequential data (like text, speech, or time series).

2.1.1 LSTM(Long Short-Term Memory)

Why LSTM?

Traditional **RNNs** suffer from the *vanishing gradient problem*, meaning they struggle to remember information from far back in a sequence. **LSTM** was introduced to solve this.

Core Idea

LSTM introduces a *cell state (memory)* and uses gates to control information flow.

Three Gates in LSTM

- **Forget Gate**: Decides what information to remove from memory.
- **Input Gate**: Decides what new information to store.
- **Output Gate**: Decides what part of memory to output.

Key Feature

It maintains:

- * Hidden state (short-term memory)
- * Cell state (long-term memory)

This structure allows LSTM to remember information over long sequences.

2.1.2 GRU (Gated Recurrent Unit)

GRU is a simplified version of LSTM.

Why GRU?

It aims to:

- Be computationally simpler
- Train faster
- Use fewer parameters
- Perform similarly to LSTM in many tasks

Two Gates in GRU

1. Update Gate Controls how much past information to keep.
2. Reset Gate Controls how much past information to forget.

2.1.3 LSTM vs GRU Comparison

Table 2.2: LSTM vs GRU Comparison

Feature	LSTM	GRU
Gates	3	2
Cell state	Yes	No
Parameters	More	Fewer
Speed	Slower	Faster
Performance	Strong for long sequence	Comparable in many cases

Key Difference from LSTM

- * No separate cell state
- * Hidden state = memory
- * Fewer gates → simpler architecture

When to Use Which?

Use LSTM when:

- Very long dependencies matter
- You want more control over memory

Use GRU when:

- You want faster training
- Dataset is smaller
- Simpler model preferred

2.1.4 AlexNet

AlexNet is a deep convolutional neural network (CNN) that revolutionized computer vision in 2012. Why AlexNet Was Important:

Before 2012:

Traditional computer vision relied on hand-crafted features.

Deep learning wasn't dominant yet.

AlexNet:

Reduced ImageNet error rate from 26% → 15%

Proved deep CNNs + GPUs work extremely well

Triggered the deep learning revolution

The Rectified Linear Unit (ReLU) is a widely used *activation function* in deep learning, defined as:

$$f(x) = \max(0, x)$$

Simple Summary

AlexNet proved deep CNNs could dominate image recognition. It started the modern deep learning era in computer vision.

2.1.5 R-CNN (Region-based CNN)

If YOLO and SSD are the "fast" sprinters of the object detection world, R-CNN (Regions with Convolutional Neural Networks) is the "slow and steady" detective.

Why it was a revolution?

Before R-CNN came out in 2014, object detection was mostly based on very simple mathematical templates. R-CNN proved that:

- **Deep Learning (CNNs)** could be used to find objects, not just classify them.
- **Transfer Learning** worked—you could take a CNN trained on ImageNet and "fine-tune" it for detection.

Problems

The reason we eventually moved to YOLO and SSD is that R-CNN was incredibly inefficient:

- **2,000 passes:** It had to run a CNN 2,000 times for every single image.
- **Time:** It took about 47 seconds to process one image on a high-end GPU. You couldn't use it for video or self-driving cars.
- **Storage:** You had to save the features of those 2,000 boxes to a hard drive during training, which took up hundreds of gigabytes.

The "Faster" Evolution

Because R-CNN was so slow, researchers kept improving it:

Table 2.3: R-CNN and its Evolutions

Model	Key Improvements	Speed
R-CNN(2014)	first to use CNNs for detection ^a	47sec/image
Fast R-CNN(2015)	processes the whole image once before cropping features	2sec/image
Faster R-CNN(2016)	replaced "Selective Search" with a dedicated AI network.	0.2sec/image

^a CNN(before R-CNN) could tell you what was in the image and its probability (92% of Dog, 4% of Cat, 1% of Car), but it had no idea where it was or how many of them. If there were three dogs in the photo, CNN would still just say Dog.

Summary

R-CNN is the grandfather of modern object detection. It is based on CNNs, but unlike the "one-stage" models (YOLO/SSD), it is a two-stage model: it finds the "where" first, and then asks the "what" later.

2.1.6 YOLO (You Only Look Once)

YOLO is a real-time object detection algorithm that detects objects in an image in a single pass rather than scanning multiple times. The name(You Only Look Once) reflects the fact that the model processes image **once** to detect everything.

Core Idea

Traditional object detectors, like R-CNN (Section 2.1.5), use two stages:

1. Generate region proposals for possible objects;
2. Classify objects within those regions.

YOLO treats detection as a single regression problem: input an image then output a bounding boxes and class probabilities for all objects.

2.1.7 SSD(Single Shot MultiBox Detector)

Released shortly after YOLO by researchers at Google/UNC, SSD was designed to fix one of YOLO's early weaknesses: small objects.

2.1.8 GPT (Generative Pre-trained Transformer)

GPT is based on the **Transformer** architecture, not RNN.

Theses about GPT

Original paper: [Attention Is All You Need](#) (introduced Transformer)

First GPT paper: [Improving Language Understanding by Generative Pre-Training](#)

Core Idea of GPT

Instead of processing sequentially like **RNN**, **GPT** uses *Self-Attention Mechanism*. Each word looks at all other words in the sentence simultaneously. So instead of using (2.1), it computes **attention weights** between all tokens.

$$h_t = f(x_t, h_{t-1}) \quad (2.1)$$

Table 2.4: Major Differences

Feature	RNN	GPT
Architecture	Recurrent	Transformer
Processing	Sequential	Parallel
Memory	Hidden state	Attention mechanism
Long-range dependency	Weak	Strong
Training speed	Slow	Fast (parallelizable)
Scalability	Limited	Extremely scalable

Intuition Example

Sentence: **The animal didn't cross the street because it was tired.**

RNN: Must carry memory **step-by-step** from beginning to *it*.

GPT: *It* directly attends to *animal* via attention weights. **No need to pass** information through many steps.

Why GPT replaced RNN in NLP(Natural Language Processing)?

- Better at long-distance relationships
- Parallel training on GPUs
- Scales extremely well (billions/trillions of parameters)
- Works for text, image, audio (multimodal)

Simple Summary

RNN = memory chain, passes information forward like a relay race.

GPT = attention graph, connects everything directly.

2.1.9 Reinforcement Learning

Think of Reinforcement Learning (RL) as the way you would train a dog: you don't give the dog a manual on "how to sit," you just give it a treat when it happens to sit, and a stern "no" when it jumps on the couch.

In AI terms, it is a type of machine learning where an Agent learns to make decisions by performing actions in an Environment to maximize a Reward.

The 4 Core Components of RL

To understand RL, you have to look at the "Loop" that happens millions of times during training:

- The **Agent**: The "student" or the AI (e.g., a robot or a character in a video game).
- The **Environment**: The world the agent lives in (e.g., a maze or a stock market).
- The **Action (A)**: What the agent chooses to do (e.g., move left, buy a stock, jump).
- The **State (S)**: The current situation (e.g., the agent's coordinates in the maze).
- The **Reward (R)**: The feedback. Positive for a good job, negative for a mistake.

How it differs from other AI

You've been asking about CNNs and RNNs — those usually fall under *Supervised Learning*. Here is the difference:

Table 2.5: Supervised vs Reinforcement learning

Supervised Learning (CNN/RNN)	Reinforcement Learning
You show the AI a picture of a cat and say, "This is a cat." It learns by matching inputs to labels.	You don't tell the AI what the "right" move is. You just tell it the goal (e.g., "Win the game"). It has to figure out the best moves through trial and error.

2.1.10 Transfer Learning

It is a technique where you take a model that has already been trained on a massive dataset (like 1.2 million photos in ImageNet) and "transfer" its knowledge to a new, smaller task. Instead of starting from zero and teaching a model what an "edge," a "circle," or a "texture" is, you start with a model that is already an expert at seeing and just teach it the final details of your specific project. In AI, if a model (like ResNet) already knows how to recognize animals, it will learn to recognize cancerous cells in X-rays much faster than a model that has never seen an image before.

How it Works (The Process)

When you use transfer learning, you typically follow these three steps:

1. **Select a Pre-trained Model**: You download a "Backbone" model that was trained by a big company (Google, Microsoft, Meta) on a huge dataset.
2. **Freeze the Base**: You "lock" the early layers of the network. These layers have already learned general features (edges, shapes, colors) that are useful for any image. You don't want to change these because they are already perfect.
3. **Replace the "Head"**: You chop off the very last layer (the part that says "This is a dog") and replace it with your own layer (e.g., "This is a rusty bolt" vs. "This is a clean bolt"). You only train this new part on your small dataset.

Summary

Transfer learning allows you to "stand on the shoulders of giants." You use the billions of dollars spent by tech giants to build your own high-quality AI with very little data.

2.1.11 SVM(support Vector Machine)

An SVM is a powerful supervised learning model used for classification and regression. If a CNN is about "seeing" and an RNN is about "remembering," an SVM is about "finding the best boundary." Imagine you have a plot of land with two types of trees: Oak and Pine. An SVM tries to draw a fence that separates them as widely as possible.

2.1.12 Ensemble Learning

Ensemble Learning is the strategy of combining multiple models to solve a single problem. The core philosophy is that "a group of specialists is smarter than a single genius". In an ensemble, you train several "base learners" (which could be SVMs, CNNs, or simple Decision Trees) and then aggregate their predictions. This reduces the risk of making a major error because one model's mistake is often corrected by the others.

Table 2.6: Comparison Between Support Vector Machines (SVM) and Deep Learning

Feature	SVM	Deep Learning (CNN/RNN/Trans.)
Data Size	Performs well on small to medium datasets	Requires massive datasets to reach peak accuracy
Interpretability	High; based on geometric margins ¹	Low; often considered a “black box”
Training Speed	Generally fast and mathematically stable	Slow; requires high-end GPUs and tuning
Feature Selection	Uses the “Kernel Trick” for non-linear data	Automates feature extraction through layers
Complexity	Struggles with raw high-res images/video	Excellent for unstructured data (images, text)

¹ Support vectors provide clear indicators of the decision boundary’s location.

2.2 What is deep learning

- Deep learning is a branch of machine learning that trains computers to do what comes naturally to humans: Learn from experience.
- Machine learning algorithms use computational methods to “learn” information directly from data without relying on a predetermined equation as a model.
- Deep learning is especially suitable for computer vision, which is important for solving problems such as face recognition, motion detection, and many advanced driver assistance technologies.

Deep Neural Networks (DNNs)

- A deep neural network combines multiple nonlinear processing layers, using simple element operating in parallel and inspired by biological nervous systems.
- A deep neural network consists of an input layer, several hidden layers, and an output layer.
- The layers are interconnected via nodes, or neurons, with each hidden layer using the output of previous layer as its input.

Table 2.7: Machine Learning (ML) vs Deep Learning (DL)

Machine Learning	Deep Learning
✓ Good results with small data sets	✗ Requires very large data sets
✓ Quick to train a model	✗ Computationally intensive
✗ Need to try different features and classifier to achieve best results	✓ Learns features and classifiers automatically
✗ Accuracy plateaus	✓ Accuracy is unlimited

2.2.1 Neural Network

Activation: This is the math used inside each neuron. *ReLU* is the modern standard and usually works best. *Sigmoid* and *Tanh* are older methods that can be slower.

Learning Rate (η): How fast the model adjusts.

- If it’s too high (e.g. 10), the model might “jump” over the solution and go crazy.
- If it’s too low (e.g. 0.00001), it will take forever to learn. **0.03** is a good starting point.

Regularization: This helps prevent the model from being too “twitchy” or over-complex.

Activation function

σ is the Activation function. Weights ($\mathbf{w}$) are per connection (each line has its own weight). Biases ($\mathbf{b}$) are per neuron (each circle has its own bias).

$$y = \sigma(\mathbf{w} \cdot x + \mathbf{b}) \quad (2.2)$$

$$\mathbf{Y} = \sigma(w_1 \mathbf{X}_1 + w_2 \mathbf{X}_2 + \dots + w_n \mathbf{X}_n + \mathbf{b}) \quad (2.3)$$

Table 2.8: Common used activation functions

Name	Function	Range
ReLU	$\sigma(x) = \max(0, x)$	$[0, x]$
tanh	$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$[-1, 1]$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	$[0, 1]$

Loss Functions

Loss functions measure how far a model's prediction is from the true value. Different tasks use different loss functions. Below are the most commonly used ones.

Table 2.9: Regression Loss Functions¹

FUNCTION	EQUATION	DESCRIPTION
Mean Squared Error (MSE)	$L = \frac{1}{n} \sum_1^n (y_i - \hat{y}_i)^2$	1)Squares the error 2)Large errors are penalized heavily 3)Very common in regression
Mean Absolute Error (MAE)	$L = \frac{1}{n} \sum_1^n y_i - \hat{y}_i $	1)Uses absolute difference 2)Less sensitive to outliers than MSE
Root Mean Squared Error (RMSE)	$L = \sqrt{\frac{1}{n} \sum_1^n (y_i - \hat{y}_i)^2}$	1)Square root of MSE 2)Same unit as the predicted variable
Huber Loss	Combines MSE and MAE.	1)Robust to outliers 2)Used in many practical ML systems

¹ Used when predicting continuous values (e.g., house price, temperature).

Note: checkout (Table 2.12)

Table 2.10: Classification Loss Functions

FUNCTION	EQUATION	DESCRIPTION
Binary Cross-Entropy ¹	$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$	logistic regression binary neural networks
Categorical Cross-Entropy ²	$L = -\sum_1^C y_i \log(\hat{y}_i)$	C = number of classes softmax output layer
Sparse Categorical Cross-Entropy		

¹ Used for binary classification.

² Used for multi-class classification.

Backpropagation

Backpropagation (backward propagation of errors) is an algorithm used to train neural networks. It calculates how much each weight in the network contributed to the prediction error and then adjusts those weights to reduce the error. In simple terms, it answers the question: "Which weights in the network should change, and by how much, to make the prediction better?"

Taylor Expansion

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n \quad (2.4)$$

why old weight subtract derivative of weight can guarantee overall loss descent?

For a small change in weight Δw , the change in loss L can be approximated (2.4) as:

$$L(w + \Delta w) \approx L(w) + \frac{\partial L}{\partial w} \Delta w$$

If we define our change Δw as the negative derivative $-\eta \frac{\partial L}{\partial w}$, and substitute it back into the equation:

$$\begin{aligned} L(w_{new}) &\approx L(w) + \frac{\partial L}{\partial w} \left(-\eta \frac{\partial L}{\partial w} \right) \\ &\approx L(w) - \eta \left(\frac{\partial L}{\partial w} \right)^2 \end{aligned}$$

Since any real number squared is always non-negative, and the learning rate η is positive, we are effectively subtracting a positive value from the current loss. Thus: $L(w_{new}) \leq L(w_{old})$

2.3 Matlab

2.3.1 Data Labelling and Augmentation

2.3.2 Implementation of Machine Learning

MATLAB makes use of four graphical tools for training neural networks to solve problems:

1. Function Fitting (nftool)
2. Pattern Recognition (nprtool)
3. Clustering (nctool)
4. Time Series (ntstool)

There are four ways to use the MATLAB Toolbox:

1. Through the four graphical user interfaces (GUIs).
2. Through basic command-line operations.
3. Through customization.
4. Through any of the functions contained in the toolbox.

2.4 Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model. It compares the actual labels with the predicted labels made by the model.

Why it is called **confusion**?

It shows where the model gets confused between classes (e.g., predicting negative when it should be positive).

Table 2.11: Confusion Matrix of a binary classification problem (two classes, e.g. positive vs negative)

	Predicted Positive	Predicted Negative	
Actual Positive	True Positive (TP) ¹	False Negative (FN) ⁴	Recall or TPR
Actual Negative	False Positive (FP) ³	True Negative (TN) ²	
	Precision or PPV		Accuracy

¹ Model correctly predicts the positive class.

² Model correctly predicts the negative class.

³ Model predicts positive but actually negative. Type I error.

⁴ Model predicts negative but actually positive. Type II error.

The **F1 Score** is a metric used to evaluate classification models. It combines **Precision** and **Recall** into a single number. It is especially useful when you want a balance between precision and recall. The F1 score is

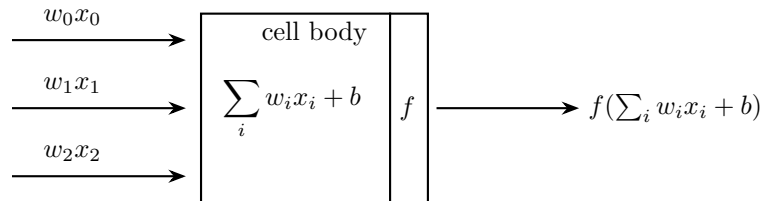
the harmonic mean (Section 7.2) of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

This means:

- If both precision and recall are high, F1 will be high.
- If either one is low, F1 will drop.

2.5 Artificial Neural Networks



2.5.1 Three Functional Operations

1. Vector weight function: Form the product $\mathbf{W} \cdot \mathbf{p}$
2. Vector input function: Form the net input $\mathbf{n} = \mathbf{W} \cdot \mathbf{p} + b$
3. Vector transfer function: Produce the scalar output $\mathbf{a} = f(\mathbf{W} \cdot \mathbf{p} + b)$

2.5.2 Feedforward Neural Networks

- Multilayer networks are capable of performing any linear or nonlinear computations, they can approximate any reasonable function arbitrarily well.
- Networks are also sensitive to the number of neurons in their hidden layers.
- Too few neurons can lead to underfitting.
- Too many neurons can contribute to overfitting, in which all training points are well fitted, but the fitting curve oscillates wildly between these points.

2.5.3 Multilayer Perceptron

Horner Algorithm

$$f(x) = \sum_{i=0}^n a_i \cdot x^i = (((a_n x + a_{n-1})x + a_{n-2})x) + \dots + a_0, a_n \neq 0$$

Kolmogorov Theorem

WIP

2.5.4 Loss Functions

Table 2.12: Common Loss functions

Loss Function	Equation
0-1 loss function	$L(Y, f(X)) = \begin{cases} 1 & Y \neq f(X) \\ 0 & Y = f(X) \end{cases}$
square loss function	$L(Y, f(X)) = (Y - f(X))^2$
absolute loss function	$L(Y, f(X)) = Y - f(X) $
logarithm loss function	$L(Y, p(Y X)) = -\log(p(Y X))$

note: checkout (Table 2.9) for more info.

2.5.5 Three Datasets

1. **Training dataset:** A set of examples are used to fit the parameters of the model. The dataset often consists of pairs of an input vector and the corresponding output vector. (70%)
2. **Test dataset:** A set of samples are used only to assess the performance (i.e., generalization) of a fully specified classifier.(15%)
3. **Validation dataset:** A dataset of samples are used to tune the hyperparameters (i.e., the architecture) of a classifier.(15%)

A dataset can be repeatedly split into a training dataset and a validation dataset, we call this **Cross Validation**.

Chapter 3

Data Mining and Machine Learning(COMP809)

3.1 How to use Jupyter Notebook in VSC

1. install python: `winget install --id python.Python.3.14 --source winget -e`
2. install necessary packages using pip:
`pip install numpy`
`pip install pandas`
`pip install seaborn`
3. install VSC: `winget install --id TeXstudio.TeXstudio --source winget -e`
4. install VSC plugins: python and jupyter
5. done

3.2 How to install Jupiter Notebook without VSC

install: `pip install notebook`

and then run: `jupyter notebook`

or run: `C:/Users/Jerry/AppData/Roaming/Python/Python314/Scripts/jupyter.exe notebook`

if something is wrong, just go to python script folder, append folder to PATH

3.3 Regression Models

We will often create models that explain how one random variable Y is affected by one or more other variables $x_1, x_2, \dots, x_p$. Y is the *response variable* and the x variables are called the *explanatory variables* (sometimes also called *predictor variables* or *covariates*). Models are used for either: **Description:** interested in better understanding the problem under study. **Prediction:** predict values of Y using the explanatory variable(s).

Why use regression models?

- They are easy to apply and interpret.
- The mathematical theory underlying them is well understood.
- They often provide an adequate approximation to reality.

3.3.1 Simple Linear Regression

We might want to model the relationship between the mean value of Y , and a single explanatory variable x using the simple linear regression model (3.1). A single neuron without activation basically performs linear regression, so simple linear regression can be viewed as the simplest neural network (one neuron).

$$E[\mathbf{Y}_i] = \beta_0 + \beta_1 \mathbf{x}_i + \epsilon_i \quad (3.1)$$

	Symbol	Description
Where:	Y_i	Dependent variable (target variable)
	x_i	Independent variable (input feature)
	β_0	Intercept of the regression model
	β_1	Slope coefficient
	ϵ_i	Random error term
	n	Number of observations

3.3.2 Parameter estimation – least squares

The method of *ordinary least squares* (OLS) involves drawing the line so that the sum of squared deviations of the original observations from the fitted line is minimized. How to choose β_0 and β_1 to minimize the value in (3.2).

$$\min \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right) \quad (3.2)$$

```

1 import pandas as pd
2 import statsmodels.formula.api as smf
3 data = pd.read_csv("data.csv")
4 model = smf.ols(formula='response ~ predictor', data=data).fit()
5 model.summary()
```

3.3.3 Model assumptions

if the following four assumptions about the errors:

1. the expected value of the errors is a constant = 0 ($E[\epsilon_i] = 0$);
2. the errors ($\epsilon_1, \dots, \epsilon_n$) are independent;
3. the error variance, $\text{var}(\epsilon_i) = \sigma^2$, is an (unknown) constant;
4. the errors ($\epsilon_1, \dots, \epsilon_n$) are normally distributed;

then the least square estimate will be: Unbiased and normally distributed, so we can calculate confidence intervals and hypothesis tests for β_0 and β_1 .

Checking model assumptions

- We can check (1) by looking for trends in residuals vs fitted values (or vs predictor variable) as a useful diagnostic.
- We can check (2) by examining residuals vs time order (if known). Long sequences of positive or negative residuals suggests that the errors are dependent.
- We can check (3) by examining the spread of residuals. The spread of residuals should not vary systematically (heteroscedasticity).
- We can check (4) using a normal Q-Q plot. Points should stay close to a straight line.

t-test and p-value

In statistics, a p-value is a tool used to help decide whether the results you see in an experiment are likely due to a real effect or just random chance.

- A low p-value (typically ≤ 0.05): This means the results you observed are very unlikely to happen if the null hypothesis were true. Because the result is so surprising under the "nothing is happening" assumption, we usually "reject" that assumption and conclude that there is a statistically significant effect.
- A high p-value (typically > 0.05): This means your results are quite likely to occur just by random chance. In this case, you "fail to reject" the null hypothesis; you don't have enough evidence to claim a real effect.

It is very common to misunderstand p-values. Here are three things p-values do not tell you:

- **They don't measure the size of the effect:** A tiny, trivial difference can have a very low p-value if your sample size is massive.
- **They don't tell you the probability that your hypothesis is true:** A p-value is about the *data*, not the *hypothesis*.
- **The 0.05 threshold is arbitrary:** It is a standard convention in science, but it isn't a law of nature. A p-value of 0.051 is not fundamentally different from 0.049, yet it often determines whether a study gets published.

3.3.4 Multiple Linear Regression

Multiple Linear Regression is a statistical method used to model the relationship between one dependent variable and multiple independent variables.

Multicollinearity

Multicollinearity is when predictors variables are correlated (i.e. a change in one x-variable is associated with a shift in another). This causes the following problems:

- Parameter estimation becomes unstable (small changes in data may lead to very different parameter estimates).
- Reduced precision of the estimates (so p-values of hypothesis tests are unreliable).

Variance Inflation Factor (VIF)

VIF (3.3) measure is often used to determine the severity of multicollinearity. It is calculated for each predictor and uses the other predictors to see how well it can be modelled.

$$\text{VIF}_j = \frac{1}{(1 - R_j^2)} \quad (3.3)$$

where the R_j^2 is the coefficient of determination for model:

$$x_1 = \beta_0 + \beta_1 x_2 + \beta_3 x_3$$

Table 3.1: Some common rules of thumb

VIF	Interpretation
VIF < 1	No correlation
1 < VIF < 5	Moderate correlation (generally acceptable)
VIF > 5	Highly correlated (potentially problematic)

DEMO

```

1 from statsmodels.tools.tools import add_constant
2 from statsmodels.stats.outliers_influence import variance_inflation_factor
3
4 X_const = add_constant(X)
5 vif_data = pd.DataFrame()
```

```

6 | vif_data["Feature"] = X_const.columns
7 | vif_data["VIF"] = [
8 |     variance_inflation_factor(X_const.values, i)
9 |     for i in range(X_const.shape[1])
10 | ]
11 | print(vif_data)

```

3.3.5 others

3.4 Non-numeric input coding

To represent non-numeric inputs in a model, you must convert them to some sort of numeric values. This conversion is most commonly done by creating design variables (or dummy variables), with each design variable representing approximately one level of the categorical input. **One-hot encoding** (Table 3.2) is one of the most popular methods (others exist). For each unique category, create a new binary column (0s and 1s). Each row gets a 1 in the column corresponding to its category and 0s elsewhere.

Table 3.2: Dataset before and after One-Hot Encoding

Original Dataset		One_Hot Encoded Dataset		
Color		Red	Green	Blue
0	Red	0 1	0	0
1	Blue	1 0	0	1
2	Green	2 0	1	0
3	Blue	3 0	0	1
4	Red	4 1	0	0

3.5 Curse of Dimensionality

3.5.1 Prerequisite

The main difference between **scikit-learn** (**sklearn**) `LinearRegression` and **statsmodels** `OLS` (all python libraries) is their purpose:

- **scikit-learn**: machine learning / prediction
 - prediction
 - model training pipelines
 - cross-validation
 - feature selection
- **statsmodels**: statistical inference / analysis
 - statistical interpretation
 - regression diagnostics
 - hypothesis testing

The Curse of Dimensionality refers to problems that occur when the number of features (dimensions) in a dataset becomes very large. As dimensionality increases, data becomes sparse, distances become less meaningful, and many machine learning algorithms perform worse. The term was introduced by Richard Bellman when studying optimization problems.

“As the number of features or dimensions grows, the amount of data we need to generalize accurately grows exponentially.”

— Charles Isbell, Professor, School of Interactive Computation, Georgia Tech

Dimensionality reduction has many advantages, such as:

- Fewer features potentially imply less complexity.
- Less storage space.
- It speeds up the analyses.

- Model accuracy improves due to less misleading data.
- It removes noise and redundant features.

So we need to reduce dimensions, there are at least two types of methods: *Feature Selection* and *Feature Extraction*.

3.5.2 Feature Selection

Filter Methods

Filtering approaches use a ranking or sorting algorithm to filter out those features that are less useful.

Wrapper Methods

The attributes subset selection is done using the learning algorithm and generally selects features by directly evaluating their impact on the performance of a model. We have those methods:

- Forward selection.

```

1      from mlxtend.feature_selection import SequentialFeatureSelector as SFS
2
3      sfs = SFS(
4          LinearRegression(),
5          k_features="best",
6          forward=True, # change to False if performing 'Backward selection'
7          floating=False,
8          scoring="r2",
9          cv=5
10     )
11     sfs.fit(X_train, y_train)
12     selected_features_fwd = list(sfs.k_feature_names_)
13     print("Forward Selection: ", selected_features_fwd)

```

- Backward selection (or backward elimination).
Same as above code, but change **forward** parameter to **False**.
- Stepwise selection (combination of forward and backward selection).

Embedded Methods

Embedded methods use algorithms that have built-in feature selection methods. For instance, Lasso.

3.5.3 Feature Extraction

Feature selection drops some redundant variables. Might lose important information that was uniquely reflected in the eliminated variables. Feature extraction means creating new features by transforming the original ones.

- Original features are combined or transformed.
- New features represent the same information in fewer dimensions.

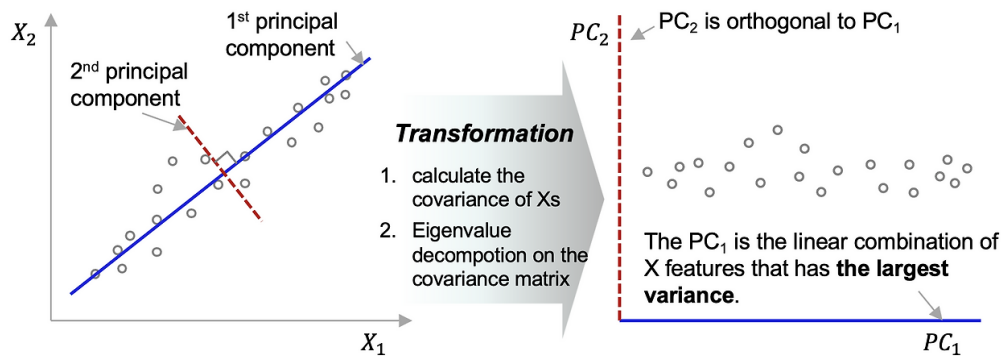
PCA (Principal Component Analysis)

To understand how Principal Component Analysis (PCA) works in a machine learning pipeline, it is important to distinguish between the training phase and the prediction (testing) phase. You must apply the same transformation to all datasets, *but the transformation parameters are derived solely from your training set*.

Because PCA finds an axis that maximizes the variance, it is typically important that all the variables are normalized or standardized. This ensures each predictor variable has the same influence in the PCA.

The axis is chosen to maximize the variability of points when they are projected onto it. This is the maximum information of the data cloud that can be found in a single axis. A second axis can be chosen that is orthogonal (at right angles) to the 1st. The 2nd axis is also chosen to maximize variation. The positions of the data on the PCA axes are known as the PCA scores. These may be used as an input variables. This has important advantages:

- Each axis is orthogonal to the others.
- This means they are all uncorrelated.
- This removes problems with multi-collinearity.

Figure 3.1: from: <https://rfriend.tistory.com/751>

Principles to do PCA:

- Do PCA on the *covariance matrix* (Table 3.3) if variables are dimensionally homogeneous and on similar scales.
- Do PCA on the *correlation matrix* (Table 3.3) if variables are not dimensionally homogeneous or are on different scales.

DEMO

Below is code, and (Figure 3.2)

```

1 % \label{lst:pca_only_on_training_data}
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4
5 STDscaler = StandardScaler()
6 X_train_standardized = STDscaler.fit_transform(X_train)
7 X_test_standardized = STDscaler.transform(X_test)
8
9 pca = PCA(n_components = 6)
10 PC_train = pca.fit_transform(X_train_standardized)
11 PC_test = pca.transform(X_test_standardized)
12 PCS = pd.DataFrame(
13     data = PC_train,
14     columns = ["pc1", "pc2", "pc3", "pc4", "pc5", "pc6"]
15 )
16 PCS['MORT'] = data['MORT']
17 sns.scatterplot(PCS, x = "pc1", y = "pc2", lw = 1)
18 plt.title("PCA Projection of Mortality Data")
19 plt.xlabel("principal component 1")
20 plt.ylabel("principal component 2")
21 plt.show()

```

Biplot

Biplot vectors can be used to give context to the PCA by show how original variables influence the PCs:

- Longer vectors indicate a stronger contribution, the angle between vectors shows correlations (smaller angles mean higher correlation).
- Vectors pointing in the same direction as a data cluster suggest that those variables are important in defining that cluster.

DEMO

Below is code, and (Figure 3.2)

```

1 # draw Biplot to indicate how much original features contribute to PC1 and PC2
2 _scaler = 6
3 pc1_components=pca.components_[0] * _scaler

```

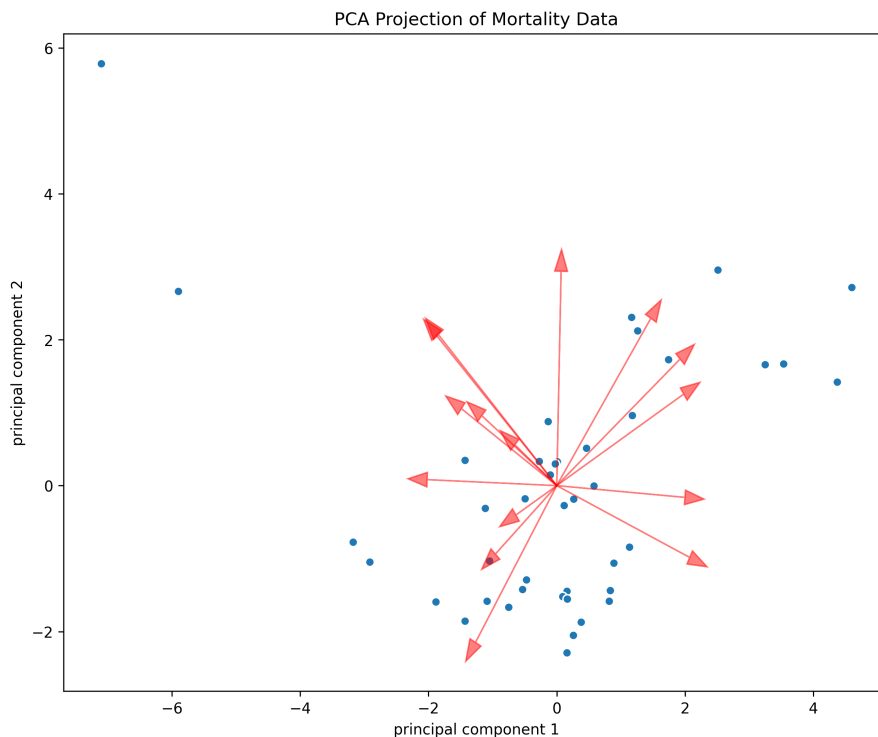


Figure 3.2: Data displayed in first two principal components, and its relations with previous features.

```

4  pc2_components=pca.components_[1] * _scaler
5
6  ordinary_feature_index = range(X.columns.shape[0])
7  for i in ordinary_feature_index:
8      plt.arrow(
9          0, 0, pc1_components[i], pc2_components[i],
10         color='r', alpha=.5, head_width=0.2
11     )
12  plt.show()

```

Table 3.3: Comparison: Covariance Matrix vs. Correlation Matrix

Feature	Covariance Matrix	Correlation Matrix
Description	Measures the absolute linear relationship between variables. It is highly sensitive to the units of measurement (e.g., if you measure distance in meters versus millimeters, the covariance values will change drastically).	Measures the standardized linear relationship. It is essentially a "rescaled" version of the covariance matrix where each variable is divided by its standard deviation.
Scale Sensitive	Yes (Depends on units)	No (Dimensionless)
Range	$(-\infty, +\infty)$	$[-1, +1]$
Interpretation	Magnitude and Direction	Strength and Direction
Use Case	Same measurement units	Different measurement units

Note: The correlation matrix is essentially the covariance matrix normalized by the standard deviations of the variables.

Shrinkage methods

Shrinkage methods are a subset of **Regularization** techniques in statistics and machine learning. The term "shrinkage" refers to the act of adding a penalty to the model's loss function, which "shrinks" the estimated coefficients toward zero (or exactly to zero). This is a powerful tool for balancing the bias-variance tradeoff, particularly when you have many features (high dimensionality) or when your features are highly correlated (multicollinearity). There are basically two primary methods: **Ridge Regression** (L_2 regularization) and **LASSO Regression** (L_1 regularization). Both methods modify the *loss function* by adding a penalty term:

Table 3.4: Normalization vs Standardization

Normalization	Standardization
Normalization rescales the values into a range of $[0,1]$.	Standardization rescales data to have a mean of 0 and standard deviation of 1. ^a
This might be useful in some cases where all parameters need to have the same positive scale. However, the outliers from the data set are obscured.	
$x_{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}}$	$x_{standardized} = \frac{x - \mu}{\sigma}$

^a after calculation, new data is normal distributed.

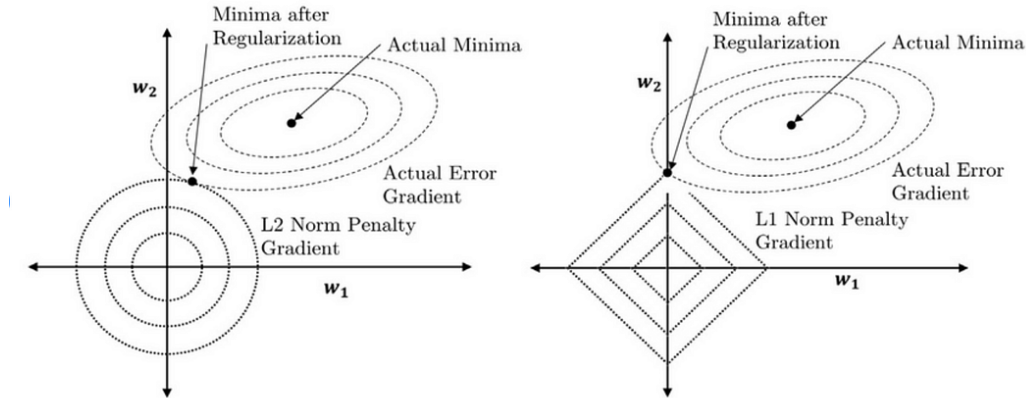


Figure 3.3: Illustration of why LASSO can set coefficients to zero.

Loss = RSS + Penalty.

- **Ridge Regression (L_2 Regularization or RR)**¹: This adds a penalty equal to the sum of the squares of the coefficients $\lambda \sum (\beta_j^2)$. It shrinks all coefficients toward zero, but they never actually reach zero. Best for when you have many features that all contribute slightly to the output, or when features are highly correlated. It keeps all variables in the model but limits their influence.

$$\sum_{i=1}^n (y_i - \hat{y})^2 = \sum_{i=1}^n (y - [\beta_0 + \sum_{j=1}^p (\beta_j x_{ij})])^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

- **LASSO Regression (L_1 Regularization)**²: This adds a penalty equal to the sum of the absolute values of the coefficients $\lambda \sum |\beta_j|$

$$\sum_{i=1}^n (y_i - \hat{y})^2 = \sum_{i=1}^n (y - [\beta_0 + \sum_{j=1}^p (\beta_j x_{ij})])^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- **The Tuning Parameter (λ)**: In both methods, the strength of the shrinkage is controlled by a hyperparameter, usually denoted as λ
 - If $\lambda = 0$: The penalty has no effect, and you get a standard OLS regression.
 - If $\lambda \rightarrow \infty$: The penalty dominates, and all coefficients shrink toward zero (making the model very simple).
 - **Selection**: We typically find the optimal λ using *Cross-Validation*.

¹<https://www.youtube.com/watch?v=Q81RR3yKn30>

²<https://www.youtube.com/watch?v=NGf0voTmlcs>

Why do we need to "shrink" coefficients?

In a standard Ordinary Least Squares (OLS) regression, the model tries to minimize the Residual Sum of Squares (RSS). If you have many predictors, OLS can become very sensitive to noise in the training data, leading to:

- **Overfitting:** The model performs perfectly on training data but fails on new data.
- **High Variance:** Small changes in the training data lead to huge changes in the estimated coefficients.

Shrinkage introduces a small amount of bias into the model to significantly reduce its variance, resulting in better overall prediction accuracy.

Table 3.5: Comparison: PCA vs. Shrinkage Methods

Criteria	PCA	Shrinkage (Lasso/Ridge)
Approach	Coordinate Transformation	Coefficient Regularization
Variables	New synthetic components	Original features preserved
Supervision	Unsupervised (ignores y)	Supervised (uses y)
Interpretability	Low	High

Note: While both reduce overfitting, PCA is a data-driven reduction while Shrinkage is a task-driven constraint.

Chapter 4

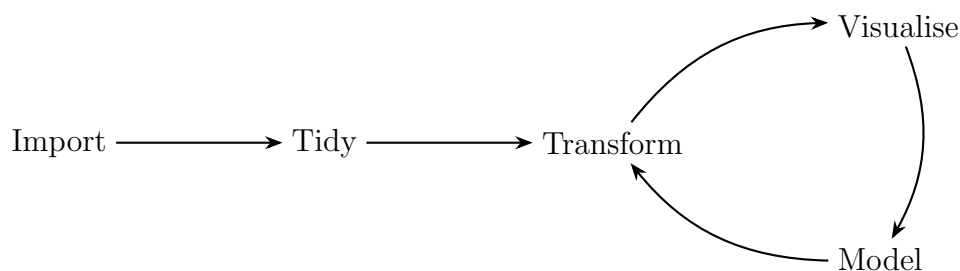
Statistical Programming for Data Sciences(COMP828)

Assignment: Hand out-Week 6 and Due date-Week 8

Project: Hand out-Week 9 and Due data-Week 13

4.1 Descriptive Statistics

- Statistics is the art of learning from data. (Probably half art, half science)
- It is concerned with the collection of data, its subsequent description and analysis, which often leads to the drawing of conclusions.
- There are two main branches of statistics:
 - Descriptive Statistics:** describing and summarizing of data
 - Inferential Statistics:** drawing of conclusions from data
- In the first few weeks, we will mainly discuss descriptive statistics.
- It is often useful to summarize a dataset using a single number that represents all the observations.
- There are many ways to do this but the three most common are **mean**, **median** and **mode**¹, which are the measures of central tendency.
- The other group of descriptive statistics, which are frequently used, are **standard deviation**, **variance**, **range**, **quartiles**, and **interquartile range**.
- Other useful statistics are proportion, **minimum**, **maximum**, etc.



4.2 Plots

- **Barplots** can only be done on qualitative variables. A barplot is a tool to visualize the distribution of a qualitative variable.
- **Histogram** is the better option to see the distribution.
- **Boxplot** summarizes the data and provides some meaningful statistics.
- **Scatter** plot is useful to visualize the relationship of two quantitative variables.

¹Mode is the most commonly occurring value in a distribution. Interestingly, there is no R function that can find the mode.

- **Line** plot is useful to visualize and summarize the key characteristics of the time series data.

```
1 barplot(table(mtcars$gear))
2 hist(mtcars$gear)
3 boxplot(mtcars$wt)
4 plot(mtcars$mpg,mtcars$wt)
5 plot(Nile)
```

```
1 mtcars[which(mtcars$am==0),] # watch ','
```

4.3 mode function in R

```
1 mode_stat <- function(x) {
2   ux <- unique(x)
3   ux[which.max(tabulate(match(x, ux)))]
4 }
```

Chapter 5

Video and Image Processing(COMP820)

5.1 Self-directed Learning: Matlab Resources for Image Enhancements

- Deblurring Images Using a Wiener Filter
- Noise Removal Links to an external site.
- Basic Image Enhancement and Analysis Techniques

5.2 Homework and self-paced learning courseware

Learn Matlab at your own pace:

[Image Processing and Computer Vision\(CV\)](#)

[Matlab Self-Paced Online Courses:](#)

5.2.1 MatLab image processing workflow



Import

You can import an image into memory and assign it to a variable in the MATLAB workspace for later manipulation, display, and export. Ending the line with a semicolon stops MATLAB from displaying the values stored in A, which is helpful because images typically contain millions of values. Image data stored in a variable A can be displayed using the `imshow` function.

You often want to view multiple images. For example, you might want to compare an image and a modified version of that image. You can display two images together using the `imshowpair` function. The "montage" option places the images A1 and A2 side by side, with A1 on the left and A2 on the right.

When you're done processing an image in MATLAB, you can save it to a file. The function `imwrite` exports data stored in the MATLAB workspace to an image file. The output file format is automatically identified based on the file extension.

```
1  A = imread("imgFile.jpg");
2  imshow(A)
3  A2 = ...;
4  imshowpair(A1, A2, "montage")
5  imwrite(imgData,"img.jpg")
```

Preprocess

Many image processing techniques are affected by image size. The images you'll be classifying all have the same size: 600 pixels tall and 400 pixels wide. You can find the size of an array by using the `size` function. The `size` function returns a vector. The first element in the vector is the number of rows in the array, which corresponds to the height of the image. The second element is the number of columns in the array, which corresponds to the width of the image.

Did the size of the array match the size of the image in pixels? Almost. You probably noticed the additional element in the vector returned by the `size` function: 3. The imported image is a color image, so that third element indicates the third dimension of the array, which stores the red, green, and blue color planes. You can access individual color planes of an image by indexing into the third dimension. For example, you can extract the green (second) color plane using the value 2 as the third index.

Most images use the unsigned 8-bit integer (`uint8`) data type, which stores integers from 0 to 255. Bright or brightly colored images contain pixel intensity values near 255 in one or more color planes. The red plane of this receipt image has some pretty bright areas. Do you think any elements of the red plane reach the maximum value of 255? You can find the largest value in an array using the `max` function. Using the "all" option finds the maximum across all values in the array. The brackets are required; they are placeholders for an unused input. Dark areas, like receipt text, contain values close to zero. Based on the image of the red plane from Task 3, do you think the red plane has any elements with a value of 0? You can find the smallest value in an array using the `min` function.

Many common tasks can be completed more quickly using functions in Image Processing Toolbox. For example, to extract all three color planes of an image array, you can use `imsplit` instead of indexing into each plane individually. You can display all three color planes at once by using the `montage` function. Try using `imsplit` to extract the color planes of `I`. Then display all three color planes using `montage`.

```
1  sz = size(A) # height * width
2  Ig = I(:, :, 2);
3  Amax = max(A, [], "all")
4  Amin = min(A, [], "all")
5  [R,G,B] = imsplit(A);
6  montage({R,G,B})
```

The algorithm you implement in this course is built around identifying text, so color is not necessary. Converting the images to grayscale eliminates color and helps the algorithm focus on the black-and-white patterns found in receipts. You can convert an image to grayscale using the `im2gray` function. If you need your converted grayscale images later, you can save them to a file. Try using `imwrite` to save `gs` to the file `gs.jpg`.

```
1  Ags = im2gray(A);
```

Why Convert to Grayscale?

- When loaded into memory, a grayscale image occupies a third of the space required for an RGB image.
- Because a grayscale image has a third of the data, it requires less computational power to process and can reduce computation time.
- A grayscale image is conceptually simpler than an RGB image, so developing an image processing algorithm can be more straightforward when working with grayscale.

Contrast and Intensity Histograms

If you are analyzing a set of images, normalizing the brightness can be an important preprocessing step, especially for identifying the black-and-white patterns of text in receipt images. An intensity histogram separates pixels into bins based on their intensity values. Dark images, for example, have many pixels binned¹ in the low end of the histogram. Bright images have many pixels binned at the high end of the histogram. The histogram often suggests where simple adjustments can be made to improve the definition of image features.

One of the essential identifying features of a receipt image is text. Receipt images should have good contrast so that the text stands out from the paper background. You can investigate the contrast in an image by viewing its intensity histogram using the `imhist` function. If the histogram shows pixels binned mainly at the high and low ends of the spectrum, then the receipt has good contrast. If not, it could probably benefit from a contrast

¹passive form of verb 'bin', means to store, to separate

adjustment. Increasing image contrast brightens brighter pixels and darkens darker pixels. You can use the `imadjust` function to adjust the contrast of a grayscale image automatically. You can, however, use the function `imlocalbrighten` to adjust the contrast of a color image.

```
1  imhist(I)
2  Iadj = imadjust(I);
3  Ibri = imlocalbrighten(I);
```

Segment

The main feature of receipt images is dark text on a bright background. How can you leverage this feature? One method is to separate the text from the background. Separating an image into distinct parts is called segmenting an image.

Intensity Thresholding

You can create a binary black-and-white image from a grayscale image by thresholding its intensity values. Values below the cutoff are assigned the value 0, while values above are assigned the value 1. When applied to arrays, logical operators like `;` and `;` generate arrays of the same size that contain logical values 1 (true) or 0 (false). You can use logical operators to threshold the intensity values of a grayscale image, creating a binary image.

The ultimate goal is to create an algorithm that can process all of our images. Unfortunately, receipt images taken in different lighting conditions have varied contrast and require different thresholds. Manually identifying a threshold using the intensity histogram works but is impractical for thousands of images. To automate the threshold selection process, you can use the `imbinarize` function, which calculates the "best" threshold for the image. In `imbinarize`, you can designate whether the foreground is bright or dark by setting the "ForegroundPolarity" option.

```
1  B = g > 200;
2  gBinary = imbinarize(g);
3  BWadapt = imbinarize(gsAdj,"adaptive","ForegroundPolarity","dark");
```

Identifying Text Patterns

You've improved the visibility of the text in a binary image. Now how can you determine if the image is a receipt? The answer lies in the pattern of text in the image.

The rows of pixels that contain text have more 0 threshold values, and the rows between lines of text have more 1s. If you sum the values across each row, the rows with text have smaller sums than the rows without text. You can compute the sums across rows of an array by using the `sum` function operating along the second dimension of the array.

```
1  # second parameter, dim = 1 sum down columns (default); dim = 2 sum across rows
2  rSum = sum(BW,2)
3  plot(rSum) # draw the sum
```

Preprocess and Postprocess

Segmentation can be improved in two ways: by preprocessing the image before binarizing and by postprocessing the binary image itself.

Spatial Filtering

Spatial filtering is a powerful technique to enhance images or their features.

- Noisy images can contain individual pixels with noticeably different intensity values than their neighbors.
- An averaging filter can smooth out these differences by replacing each pixel's value with the average of itself and neighboring pixels.
- A filter is a matrix that acts as a mask, creating a window around a pixel to perform operations.
- Applying a filter involves performing operations like weighted averages on the window, resulting in a new filtered image.
- Different filters serve different purposes, such as blurring edges or removing noise.

Images taken in low light often become noisy due to the increase in camera sensitivity required to capture the image. This noise can interfere with receipt identification by polluting regions in the binarized image. To reduce the impact of this noise on the binary image, you can preprocess the image with an averaging filter. Use the `fspecial` function to create an `n-by-n` averaging filter. You can apply a filter `F` to an image `I` by using the `imfilter`

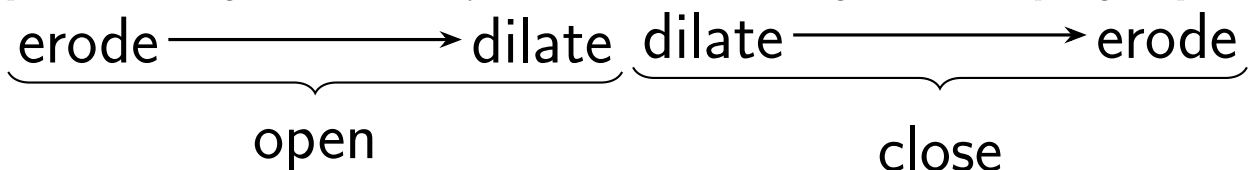
function. If you look closely, you can see that filtering introduced a small, unwanted dark outline to parts of the binary image border. The reason is that the default setting of `imfilter` sets pixels outside the image to zero. To adjust this behavior, use the "replicate" option. Instead of zeros, the "replicate" option uses pixel intensity values on the image border for pixels outside the image.

```
1 F = fspecial("average", n)
2 Ifltr = imfilter(I,F);
3 Ifltr = imfilter(I,F,"replicate");
```

Isolate the Image Background

Receipt images that have a busy background are more difficult to classify because artifacts pollute the row sum. To mitigate this issue, you can isolate the background and then remove it by subtraction.

In a receipt image, the background is anything that is not text, so isolating the background can be interpreted as removing the text. One way to remove text from an image is to use morphological operations.



Morphological Operations

Morphological operations are sliding window operations that use a structuring element to determine which neighboring pixels to include for each calculation.

- You can change the size and shape of the structuring element to affect the output of morphological operations.
- Dilation takes the brightest pixel in the window, and erosion takes the darkest.
- Opening (erosion followed by dilation) emphasizes darker parts of an image and removes smaller bright areas.
- Closing (dilation followed by erosion) emphasizes brighter parts of an image and removes smaller dark areas.

In a receipt image, the background is anything that is not text. By using a structuring element larger than a typical letter, each "window" captures what's behind the text and not the text itself. You can create a structuring element by using the `strel` function.

The foreground (the text) is dark, and the background is bright. Closing the image emphasizes and connects the brighter background, removing the thin dark lettering. Larger background objects, like the thumb, remain. The final result is an image of the background without the text. To perform a closing operation on an image `I` with structuring element `SE`, use the `imclose` function.

Now that you've isolated the background `Ibg`, you can remove it from `gs`. In general, you can remove one image from another by subtracting it. Because `imclose` emphasizes brightness in an image, the background image `Ibg` has higher intensities than `gs`. Subtracting `Ibg` from `gs` would result in negative intensities.

The sequence of operations used here is very common: closing an image, then subtracting the original image from the closed image. It's known as the "bottom hat transform," and you can use the `imbothat` function in MATLAB to perform it.

```
1 SE = strel("diamond",5)
2 Iclosed = imclose(I,SE);
3 Isub = I2 - I1;
4 BWsub = ~ ib;
5 I = imbothat(I,Se);
```

Classification and Batch Processing

what is "Local Minima" in pictures in Matlab

A local minimum usually refers to a point in the image where a function (like brightness or intensity) is lower than its neighbors.

$$I(x_0, y_0) \leq I(x, y) \quad \text{for all neighbors } (x, y)$$

Matlab DataStore The `imageDatastore` function creates an image datastore for all the image files in a folder but won't load them into memory until they are requested. You can access the datastore's properties by using a period (.). The `readimage` function loads the `nth` image from an image datastore into the MATLAB workspace.

```

1 ds = imageDatastore("localFolder")
2 folders = ds.Folders
3 files = ds.Files
4 img = read(ds); # like c style, index might be loop from 0 to len-1
5 img = readimage(ds,n); # n>0

```

```

1 nFiles = numel(ds.Files);
2 isReceipt = false(1,nFiles);
3 for i=1:nFiles
4     I = readimage(ds,i);
5     isReceipt(i) = classifyImage(I);
6 end
7 receiptFiles = ds.Files(isReceipt)
8 montage(receiptFiles)

```

5.2.2 Conclusion

Table 5.1: Common MATLAB Image Processing Functions

Function	Description	Example
<code>imread</code>	Read an image into the workspace.	<code>imread("img001.png");</code>
<code>imshow</code>	Display an imported image.	<code>imshow(I)</code>
<code>imshowpair</code>	Display two images as an overlay or montage.	<code>imshowpair(I1,I2,"montage")</code>
<code>montage</code>	Display multiple images next to each other.	<code>montage({I1, I2, I3})</code>
<code>im2gray</code>	Convert an RGB image to grayscale.	<code>im2gray(I);</code>
<code>imhist</code>	Display a histogram of a grayscale image.	<code>imhist(gs)</code>
<code>imadjust</code>	Adjust the contrast of an image.	<code>imadjust(gs);</code>
<code>imbinarize</code>	Create a binary image from a grayscale image.	<code>imbinarize(gs);</code>
<code>imresize</code>	Resize an image to half of its origin.	<code>imresize(I, 0.5);</code>
<code>imcrop</code>	Crop an image to get smaller size.	<code>imcrop(I,[xmin,ymin,width,height]);</code>
<code>fspecial</code>	Create a filter from predefined filter types.	<code>fspecial("disk",5);</code>
<code>imfilter</code>	Apply filter H to an image.	<code>imfilter(I,H);</code>
<code>strel</code>	Create a morphological structuring element.	<code>strel("rectangle",[5 5]);</code>
<code>imclose</code>	Morphological closing using SE.	<code>imclose(I,SE);</code>
<code>imopen</code>	Morphological opening using SE.	<code>imopen(I,SE);</code>
<code>imageDatastore</code>	Create datastore for image data.	<code>imageDatastore("myFolder");</code>
<code>readimage</code>	Read the <code>nth</code> image from datastore.	<code>readimage(ds,n);</code>

Note: Functions are from MATLAB Image Processing Toolbox.

5.2.3 demo

```

1 # matlab online
2 turtleVideo = VideoReader("turtles.avi")
3 #Remember to use a semicolon (;) so the values in frame are not displayed.
4 frame=readFrame(turtleVideo);
5 imshow(frame)
6 t=turtleVideo.CurrentTime
7 frame2=readFrame(turtleVideo);
8 t2=turtleVideo.CurrentTime
9 imshowpair(frame,frame2)
10
11 frameMid=read(turtleVideo,50);
12 imshow(frameMid)
13 tMid=turtleVideo.CurrentTime
14
15 # like C, continue from frame 50,
16 frameNext=readFrame(turtleVideo);

```

```

17 | imshow(frameNext)
18 | tNext=turtleVideo.CurrentTime
19 |
20 | # last frame;
21 | frameLast=read(turtleVideo,Inf);
22 | imshow(frameLast)
23 | tLast=turtleVideo.CurrentTime
24 |
25 | # rewind
26 | turtleVideo.CurrentTime=0

```

5.3 Lecture 3

Table 5.2: SWOT

internal factors		external factors	
Strengths	Weaknesses	Opportunities	Threads
things you do well	limitations or disadvantages	growth or improvement	risks or challenges
• brand reputation • skilled employees • advanced technology • good financial res.	• lack of experience • outdated technology • poor marketing • limited budget	• new market demand • tech. breakthroughs • government support • increasing popularity of AI	• strong competitors • economic downturn • new regulations • rapid tech. changes

SWOT is a simple framework used in strategic planning to analyze a situation, project, company, or idea.

blue-sky research

Also called **pure research** or **basic research** refers to research done without a specific practical goal or immediate commercial application in mind. The main purpose is to expand knowledge and understanding, even if the usefulness is not clear at the time.

5.4 Other Learning Resources

[AI Basketball Analysis](#)

[AI-basketball-analysis-google-colab.ipynb](#)

Chapter 6

Generative AI(COMP7??)

Generative AI(Gen AI) refers to models that learn patterns from data and then generate new content that resembles the training data. Instead of only classifying or predicting (like traditional machine learning), generative models create:

- Text
- Audio
- Video
- Code
- Images
- 3D models

6.1 Transformer-Based Models (LLMs)

- They are mainly used for:
- Text generation
- Code generation
- Question answering
- Chatbots
- Translation

Examples of systems:

ChatGPT

Gemini

Claude

Core idea: They predict the next token given previous tokens:

$$P = (x_t | x_1, x_2, \dots, x_{t-1})$$

6.2 GANs (Generative Adversarial Networks)

Introduced by Ian Goodfellow in 2014.

Generator → creates fake data

Discriminator → tries to detect fake vs real

Used for:

Image synthesis

Deepfakes

Style transfer

6.3 Diffusion Models

Popular in modern image generation.

Examples:

Stable Diffusion

DALL·E

Core idea:

Gradually add noise to real images.

Train model to reverse the noise process.

Generate new images by starting from noise and denoising step by step.

6.4 VAEs (Variational Autoencoders)

Architecture:

Encoder $\rightarrow$ compress input to latent vector

Decoder $\rightarrow$ reconstruct from latent space

They are more mathematically structured than GANs but usually produce blurrier outputs. After the input is compressed into a latent vector, what exactly happens?

6.4.1 Important Correction First

In a normal **autoencoder**: $x \rightarrow z \rightarrow \hat{x}$

The encoder outputs a single deterministic vector z . But in a Variational Autoencoder (VAE), the encoder does not output a single latent vector. It outputs two vectors: $\mu(x)$ $\sigma(x)$. These define a distribution:

$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x)) \quad (6.1)$$

So the encoder produces a **mean** and **variance**, not just a compressed code.

6.4.2 Step-by-Step: What Happens After Compression

Step 1 — Sampling (Reparameterization Trick)

Instead of directly using μ as latent vector, we sample noise: $\epsilon \sim \mathcal{N}(0, 1)$, then construct latent vector: $z = \mu + \sigma \cdot \epsilon$. This is called the *reparameterization trick*, if sampling directly from $\mathcal{N}(\mu, \sigma^2)$ would break backpropagation, by rewriting it this way, gradients can flow through μ and σ .

Step 2 — Decoder Takes z

Now the sampled latent vector z is fed into the decoder network.

Step 3 — Two Loss Components

VAE does NOT only minimize reconstruction error.

Chapter 7

Mathamatics

7.1 norm

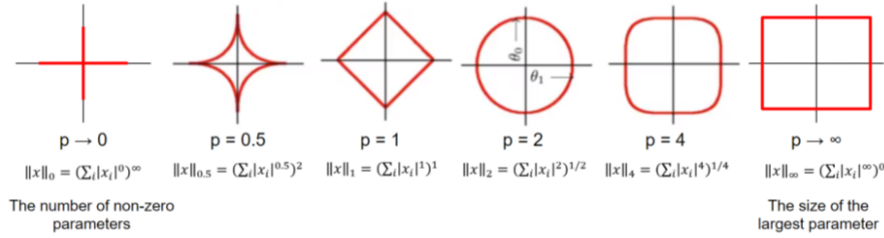


Figure 7.1: L_n norm illustrated in a 2D space.

7.2 Harmonic Mean

The Harmonic Mean is a specialized type of average that is used primarily when dealing with rates, ratios, or speed. It is one of the three "Pythagorean means," alongside the Arithmetic Mean and the Geometric Mean. For a set of n positive numbers $x_1, x_2, \dots, x_n$, the Harmonic Mean (H) is defined as the reciprocal of the arithmetic mean of the reciprocals:

$$H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}} \quad (7.1)$$

Unlike the arithmetic mean, the harmonic mean is heavily influenced by **small values**. If either Precision or Recall is very low, the F1-Score drops significantly. This forces the model to perform well on both metrics rather than relying on one high score to mask a failing one.

Table 7.1: Comparison of Pythagorean Means

Mean Type	Formula	Best Used For
Arithmetic Mean	$\frac{1}{n} \sum_{i=1}^n x_i$	General data, linear sets
Geometric Mean	$(\prod_{i=1}^n x_i)^{1/n}$	Growth rates, percentages
Harmonic Mean	$\frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$	Rates, ratios, speed, F1-score

Note: For any set of positive numbers, the relationship Arithmetic $\geq$ Geometric $\geq$ Harmonic always holds.

7.3 Marginal Probability

In probability theory, marginal probability is the probability of one variable occurring regardless of the values of other variables. Mathematically, it is obtained by summing or integrating over the other variables in a joint distribution. For example, if we have a joint probability $P(X, Y)$, the marginal probability of X is:

$$P(X) = \sum_y P(X, y)$$

The above formula is for discrete variables, as for continuous ones, we shall use calculus to sum others up:

$$P(X) = \int P(X, y) \, dy$$

7.4 Conditional Probability

Conditional probability describes the probability of an event given that another event has already occurred.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Table 7.2: Basic probability concepts

Concept	Meaning	Description
Joint probability	$P(A \cap B)$	probability that A and B occur together
Marginal probability	$P(A)$	probability of A
Conditional probability	$P(A \mid B)$	probability that A occurs given B has occurred
Independence	$P(A \mid B) = P(A)$	